Resenha: On the Criteria To Be Used in Decomposing Systems into Modules (Parnas)

Modularização por ocultamento de informação e critérios para sistemas mais evolutivos

Referência

PARNAS, David L. **On the Criteria To Be Used in Decomposing Systems into Modules**. *Communications of the ACM*, v. 15, n. 12, p. 1053-1058, dez. 1972. DOI: 10.1145/361598.361623.

Fonte 1. Referências conferidas em 10 out. 2026.

1. Introdução

O artigo de David L. Parnas discute um problema que continua central em engenharia de software: como decompor um sistema em módulos realmente úteis para desenvolvimento, manutenção e evolução. A crítica do autor é dirigida à prática de dividir o sistema de acordo com as etapas visíveis do processamento, algo que parece natural em um primeiro momento, mas que frequentemente produz forte dependência entre partes diferentes do software. Como alternativa, Parnas propõe que a modularização seja guiada pelo ocultamento de informações, isto é, pela decisão de esconder em cada módulo uma escolha de projeto que possa mudar no futuro. A tese principal é que bons módulos não representam apenas passos de execução, mas fronteiras de conhecimento e responsabilidade, o que torna o sistema mais flexível, mais compreensível e menos frágil diante de mudanças.

2. Síntese das ideias principais

Parnas organiza a discussão a partir de um pequeno exemplo, o sistema KWIC, para comparar duas formas de decomposição. Com isso, ele mostra que a qualidade da modularização depende menos da quantidade de módulos e mais do critério usado para definir suas fronteiras.

2.1 Benefícios esperados da modularização

O texto afirma que modularizar bem deve gerar três ganhos concretos. O primeiro é gerencial: equipes diferentes podem trabalhar com menos necessidade de comunicação constante. O segundo é a flexibilidade do produto: mudanças importantes em um módulo não deveriam obrigar alterações espalhadas pelo restante do sistema. O terceiro é a compreensibilidade: o software pode ser estudado módulo por módulo, o que melhora tanto o projeto quanto a manutenção. Esses benefícios, porém, não surgem apenas porque o sistema foi dividido em partes; eles dependem de uma divisão que realmente isole decisões relevantes.

2.2 O limite da decomposição por fluxo de processamento

Na primeira solução apresentada para o KWIC, a estrutura do sistema segue o fluxo de execução: entrada, rotação circular, ordenação alfabética, saída e controle principal. Essa abordagem parece intuitiva porque acompanha a sequência do trabalho realizado pelo programa. O problema é que decisões internas, como representação de linhas, forma de armazenamento, convenções de acesso e detalhes de ordenação, acabam atravessando vários módulos ao mesmo tempo. Assim, qualquer mudança nessas decisões se propaga pelo sistema e reduz a independência entre as partes. O resultado é uma organização aparentemente clara, mas estruturalmente frágil.

2.3 Ocultamento de informações como critério mais forte

Na segunda decomposição, os módulos passam a encapsular decisões de projeto, e não apenas etapas do processamento. Em vez de organizar o sistema pela ordem de execução, Parnas propõe módulos como armazenamento de linhas, entrada, circular shifter, alphabetizer e saída, cada um com interfaces próprias que escondem detalhes internos. Esse desenho permite que decisões voláteis, como estrutura de dados ou estratégias de acesso, fiquem protegidas do restante do sistema. A consequência é um software mais robusto à mudança, porque aquilo que tende a variar permanece isolado atrás de interfaces mais abstratas.

2.4 Interfaces também podem revelar informação demais

O artigo não trata information hiding como um rótulo automático. O próprio autor mostra que sua segunda solução ainda tem uma limitação importante: a interface do módulo de circular shifts revela uma ordem específica para os deslocamentos. Isso significa que, mesmo em uma decomposição melhor, ainda é possível expor detalhes demais e reduzir a liberdade futura de implementação. A lição aqui é que modularizar bem não consiste apenas em criar módulos com nomes abstratos, mas em definir interfaces que revelem somente o necessário para uso correto do serviço.

2.5 Flexibilidade, hierarquia e eficiência

Parnas também argumenta que uma boa decomposição não deve ser confundida com hierarquia, embora as duas coisas possam coexistir. Uma estrutura hierárquica baseada em dependências traz vantagens adicionais, como reaproveitamento de módulos de níveis mais baixos e possibilidade de reutilização em outros sistemas. Ao mesmo tempo, o texto reconhece uma tensão prática: interfaces mais abstratas podem gerar implementações menos eficientes se forem traduzidas de forma ingênua para subrotinas e chamadas custosas. Ainda assim, o autor sustenta que o desenho conceitual deve priorizar a qualidade arquitetural, enquanto os problemas de desempenho devem ser resolvidos no nível da implementação e das ferramentas.

3. Aplicação imaginada dos conceitos

Em um sistema de agendamentos com integrações externas, a decomposição por fluxo poderia gerar módulos como criar agendamento, validar horário, enviar notificação, salvar dados e exibir agenda. Essa organização parece limpa, mas tende a espalhar decisões importantes entre muitas partes do sistema, como formato de datas, política de disponibilidade, tratamento de reservas pendentes, regras de bloqueio e integração com provedores externos. Quando surge a necessidade de trocar o provedor de calendário, alterar a regra de janelas disponíveis ou introduzir reservas provisórias com expiração, o impacto deixa de ser localizado e passa a atravessar várias camadas.

Seguindo a proposta de Parnas, uma decomposição mais forte esconderia essas decisões em módulos próprios, como AvailabilityPolicy, CalendarGateway, PendingBookingStore, NotificationGateway e BookingRepository. Os casos de uso ficariam responsáveis por orquestrar esses serviços, sem conhecer a representação interna de cada um. Na prática, isso melhoraria a testabilidade, reduziria o acoplamento e permitiria evolução incremental com menos retrabalho. A aplicação mais importante da leitura é perceber que os módulos mais valiosos não são os que apenas acompanham o fluxo da tela ou da API, mas os que isolam aquilo que tem maior chance de mudar.

4. Conclusão

A principal contribuição do artigo está em mudar a pergunta central da modularização. Em vez de pensar apenas em como dividir o processamento em partes menores, Parnas mostra que o critério mais importante é descobrir quais decisões de projeto precisam ser protegidas das demais partes do sistema. Essa mudança de perspectiva continua atual porque conecta arquitetura a manutenção real, e não apenas à organização inicial do código. O texto também deixa claro que boas interfaces exigem disciplina e que ganhos arquiteturais não eliminam preocupações com eficiência. Mesmo assim, a ideia de ocultamento de informações permanece como uma das bases mais sólidas para projetar software capaz de evoluir sem se tornar rígido e frágil.